

Implementation Guide

Persona-Based Multi-Agent Architecture — Google ADK + PostgreSQL

Table of Contents

1. [Project Structure](#)
 2. [Writing Use Case MD Files](#)
 3. [Parsing Use Case MD Files](#)
 4. [Step 1 — Build the Tools Registry](#)
 5. [Step 2 — Build the Skills Registry](#)
 6. [Step 3 — Build the Use Cases Registry](#)
 7. [Step 4 — Build the Personas Registry](#)
 8. [Step 5 — Build the Use Case Orchestrator](#)
 9. [Step 6 — Build the Agent Orchestrator \(Agent Factory\)](#)
 10. [Step 7 — Build the Persona Orchestrator](#)
 11. [Step 8 — End-to-End Data Flow](#)
 12. [Step 9 — MD File Parser](#)
 13. [Testing and Validation](#)
 14. [Deployment Checklist](#)
-

1. Project Structure

```
project/  
├── use_cases/                # Use case MD files (input)
```

```

|   ├── edr_qa.md
|   └── rro_qa.md
|
|── skills/                                # Agent Skills standard directories
|   ├── fetch-source-data/
|   |   ├── SKILL.md
|   |   └── assets/
|   |       ├── input-schema.json
|   |       └── output-schema.json
|   ├── compare-two-sources/
|   |   ├── SKILL.md
|   |   └── assets/
|   |       ├── input-schema.json
|   └── verdict-summary/
|       └── SKILL.md
|
|── orchestrators/
|   ├── use_case_orchestrator.py          # Level 2 orchestrator
|   └── persona_orchestrator.py          # Level 1 orchestrator
|
|── parsers/
|   └── md_parser.py                      # Parses use case MD files
|
|── registry/
|   ├── tools_registry.py
|   ├── skills_registry.py
|   ├── use_cases_registry.py
|   └── personas_registry.py
|
|── db/
|   ├── schema.sql                       # Full PostgreSQL schema
|   └── connection.py
|
└── main.py                              # Entry point

```

2. Writing Use Case MD Files

A use case MD file describes a workflow in plain language. The parser reads this file and generates the corresponding use case record in PostgreSQL. Each workflow step maps to a skill from the skills registry.

Format

```

---
use_case_name: <slug>
description: <what this use case does>
input_schema:
  required: [field1, field2]
  properties:
    field1:
      type: string
      label: "Human-readable label"
---

## Workflow

### Step 1: <Step Title>
- skill: <skill-name>
- tool: <tool-name>
- description: <what this step does>
- input_mapping:
    param1: "{{ input.field1 }}"
    param2: "static_value"
- output_key: step1_result

### Step 2: <Step Title>
- skill: <skill-name>
- description: <what this step does>
- input_mapping:
    data: "{{ outputs.step1_result }}"
- output_key: step2_result

### Verdict
- skill: verdict-summary
- input_mapping:
    comparison_result: "{{ outputs.step2_result }}"
- output_key: final_verdict

```

Example: EDR QA Use Case MD File (`use_cases/edr_qa.md`)

```

---
use_case_name: edr-qa
description: >
  QA validation for EDR (Electronic Data Records). Fetches EDR data
  from the internal database and the source API, compares them field
  by field, and produces a pass/fail verdict.
input_schema:
  required: [account_number]

```

```

    properties:
      account_number:
        type: string
        label: "EDR Account Number"
---

## Workflow

### Step 1: Fetch EDR from Database
- skill: fetch-source-data
- tool: postgres_query
- description: Fetch EDR records from internal Postgres database for the given ac
- input_mapping:
    connection_string: "{{ secrets.edr_db_connection }}"
    query: "SELECT record_id, field_a, field_b, status FROM edr_records WHERE acc
    params: ["{{ input.account_number }}" ]
- output_key: edr_source_db

### Step 2: Fetch EDR from Source API
- skill: fetch-source-data
- tool: rest_api_get
- description: Fetch EDR records from the upstream source API
- input_mapping:
    url: "https://api.edr-system.com/records?account={{ input.account_number }}"
    headers:
        Authorization: "Bearer {{ secrets.edr_api_token }}"
- output_key: edr_source_api

### Step 3: Compare EDR Records
- skill: compare-two-sources
- description: Compare DB records vs API records field by field
- input_mapping:
    source_a: "{{ outputs.edr_source_db }}"
    source_b: "{{ outputs.edr_source_api }}"
    key_field: "record_id"
- output_key: edr_comparison

### Verdict
- skill: verdict-summary
- input_mapping:
    comparison_result: "{{ outputs.edr_comparison }}"
    use_case_name: "EDR QA"
- output_key: edr_verdict

```

Example: RRO QA Use Case MD File (`use_cases/rro_qa.md`)

```

---
use_case_name: rro-qa
description: >
    QA validation for RRO (Revenue Reporting Objects). Fetches property
    data for a list of property IDs from two sources and validates consistency.
input_schema:
    required: [property_ids]
    properties:
        property_ids:
            type: array
            items:
                type: string
            label: "RRO Property IDs"
---

## Workflow

### Step 1: Fetch RRO from Database
- skill: fetch-source-data
- tool: postgres_query
- description: Fetch RRO records from internal database for provided property IDs
- input_mapping:
    connection_string: "{{ secrets.rro_db_connection }}"
    query: "SELECT property_id, revenue, period, status FROM rro_records WHERE pr
    params: ["{{ input.property_ids }}" ]
- output_key: rro_source_db

### Step 2: Fetch RRO from External System
- skill: fetch-source-data
- tool: rest_api_get
- description: Fetch RRO records from external reporting system
- input_mapping:
    url: "https://api.rro-system.com/properties"
    headers:
        Authorization: "Bearer {{ secrets.rro_api_token }}"
        Content-Type: "application/json"
    body:
        property_ids: "{{ input.property_ids }}"
- output_key: rro_source_api

### Step 3: Compare RRO Records
- skill: compare-two-sources
- description: Compare DB vs API property records
- input_mapping:
    source_a: "{{ outputs.rro_source_db }}"
    source_b: "{{ outputs.rro_source_api }}"

```

```

        key_field: "property_id"
-   output_key: rro_comparison

### Verdict
-   skill: verdict-summary
-   input_mapping:
        comparison_result: "{{ outputs.rro_comparison }}"
        use_case_name: "RRO QA"
-   output_key: rro_verdict

```

Notice that both use cases reuse the **same three skills**: `fetch-source-data`, `compare-two-sources`, and `verdict-summary`. Only the tool configuration and parameter values differ.

3. Parsing Use Case MD Files

The MD parser reads a use case file and produces a structured dictionary ready to insert into PostgreSQL.

`parsers/md_parser.py`

```

import yaml
import re
from typing import Any

def parse_use_case_md(filepath: str) -> dict:
    """
    Parses a use case MD file into a structured use case dict.
    Returns a dict ready to insert into the use_cases table.
    """
    with open(filepath, "r") as f:
        content = f.read()

    # Split frontmatter and body
    parts = content.split("---", 2)
    if len(parts) < 3:
        raise ValueError(f"Invalid use case MD: missing frontmatter in {filepath}")

    frontmatter = yaml.safe_load(parts[1])
    body = parts[2]

    # Parse workflow steps from body
    steps = _parse_workflow_steps(body)

```

```

return {
    "use_case_name": frontmatter["use_case_name"],
    "description": frontmatter["description"],
    "input_schema": frontmatter.get("input_schema", {}),
    "flow": steps
}

def _parse_workflow_steps(body: str) -> list[dict]:
    """
    Parses ### Step N: Title and ### Verdict sections into flow steps.
    """
    steps = []
    # Match each ### Step or ### Verdict block
    pattern = re.compile(
        r"###\s+(Step\s+(\d+) |Verdict):\s*(.+?) (?:\n###|\Z)",
        re.DOTALL
    )

    for match in pattern.finditer(body):
        is_verdict = match.group(1).lower() == "verdict"
        step_num = int(match.group(2)) if not is_verdict else 999
        title = match.group(3).strip()
        block = match.group(0)

        step = {
            "step": step_num,
            "title": title,
            "is_verdict": is_verdict,
            "skill": _extract_field(block, "skill"),
            "tool": _extract_field(block, "tool"),
            "description": _extract_field(block, "description"),
            "input_mapping": _extract_mapping(block, "input_mapping"),
            "output_key": _extract_field(block, "output_key"),
        }
        steps.append(step)

    return sorted(steps, key=lambda x: x["step"])

def _extract_field(block: str, field: str) -> str | None:
    match = re.search(rf"^\s+{field}:\s+(.+)$", block, re.MULTILINE)
    return match.group(1).strip() if match else None

def _extract_mapping(block: str, field: str) -> dict:
    """Extract indented key-value mapping block."""

```

```

match = re.search(
    rf"^-\s+{field}:\s*\n(?:\s{{4,}}.+\\n?)+",
    block, re.MULTILINE
)
if not match:
    return {}
return yaml.safe_load(match.group(1)) or {}

```

Usage

```

from parsers.md_parser import parse_use_case_md
from registry.use_cases_registry import upsert_use_case

use_case = parse_use_case_md("use_cases/edr_qa.md")
await upsert_use_case(use_case)

use_case = parse_use_case_md("use_cases/rro_qa.md")
await upsert_use_case(use_case)

```

4. Step 1 — Build the Tools Registry

PostgreSQL Schema

```

CREATE TABLE tools (
    tool_id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tool_name        VARCHAR(100) UNIQUE NOT NULL,
    tool_type        VARCHAR(50)  NOT NULL,
    description       TEXT,
    config_schema     JSONB NOT NULL,
    created_at       TIMESTAMP DEFAULT NOW()
);

```

Seed Tools

```

INSERT INTO tools (tool_name, tool_type, description, config_schema) VALUES
(
    'postgres_query',
    'database',
    'Executes a parameterized SQL query against a PostgreSQL database. Use for any
    '{
        "required": ["connection_string", "query"],

```



```

        "properties": {
            "connection_string": {"type": "string"},
            "query": {"type": "string"},
            "params": {"type": "array", "default": []}
        }
    }'
),
(
    'rest_api_get',
    'api',
    'Makes an HTTP GET or POST request to a REST API endpoint. Use for any external
    '{
        "required": ["url", "headers"],
        "properties": {
            "url": {"type": "string"},
            "headers": {"type": "object"},
            "body": {"type": "object"},
            "timeout_seconds": {"type": "integer", "default": 30}
        }
    }'
);

```

Python Tool Loader

```

# registry/tools_registry.py

async def get_tool(tool_name: str, db) -> dict:
    return await db.fetchrow(
        "SELECT * FROM tools WHERE tool_name = $1", tool_name
    )

def build_adk_tool(tool_def: dict, bound_params: dict):
    """
    Wraps a tool definition + bound params into a callable ADK tool.
    """
    from google.adk.tools import FunctionTool

    async def execute(**kwargs):
        merged = {**bound_params, **kwargs}
        if tool_def["tool_type"] == "database":
            return await run_postgres_query(merged)
        elif tool_def["tool_type"] == "api":
            return await run_rest_api(merged)

    return FunctionTool(
        name=tool_def["tool_name"],

```

```

        description=tool_def["description"],
        func=execute
    )

```

5. Step 2 — Build the Skills Registry

PostgreSQL Schema (Agent Skills Standard Aligned)

```

CREATE TABLE skills (
    skill_id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    -- L1: Loaded at startup (name + description only)
    skill_name        VARCHAR(100) UNIQUE NOT NULL, -- lowercase-hyphenated
    description        TEXT NOT NULL,                -- max 1024 chars, WHAT + WHE

    -- L2: Loaded when skill is activated
    instruction        TEXT NOT NULL,                -- SKILL.md body content
    allowed_tools      TEXT,                          -- space-separated tool names
    compatibility       VARCHAR(500),

    -- L3: Loaded on demand
    input_schema       JSONB,
    output_schema      JSONB,
    tool_id            UUID REFERENCES tools(tool_id),

    -- Standard metadata
    metadata           JSONB DEFAULT '{}',
    license            VARCHAR(200),
    created_at         TIMESTAMP DEFAULT NOW()
);

```

Seed Skills

```

-- Skill 1: fetch-source-data
INSERT INTO skills (skill_name, description, instruction, allowed_tools, metadata
(
    'fetch-source-data',
    'Fetches structured data from a configured database or API tool. Use when an ag
E'## Instructions\n\nYou are a precise data retrieval agent. Your only job is t
'postgres_query rest_api_get',
    '{"author": "qa-platform-team", "version": "1.0"}'
),

```

```

-- Skill 2: compare-two-sources
(
    'compare-two-sources',
    'Compares two datasets field by field and returns a structured diff. Use when t
E'## Instructions\n\nYou are a meticulous data comparison agent.\n\n### Steps\n
NULL,
    '{"author": "qa-platform-team", "version": "1.0"}'
),

-- Skill 3: verdict-summary
(
    'verdict-summary',
    'Summarizes comparison results into a human-readable report and delivers a fina
E'## Instructions\n\nYou are an impartial QA verdict agent.\n\n### Steps\n1. Re
NULL,
    '{"author": "qa-platform-team", "version": "1.0"}'
);

```

Python Skills Loader (Progressive Disclosure)

```

# registry/skills_registry.py

async def get_skills_index(db) -> list[dict]:
    """L1: Load name + description only for all skills."""
    return await db.fetch("SELECT skill_name, description FROM skills")

async def get_skill_instructions(skill_name: str, db) -> dict:
    """L2: Load full instructions when skill is activated."""
    return await db.fetchrow(
        "SELECT skill_name, instruction, allowed_tools, compatibility FROM skills
        skill_name
    )

async def get_skill_schemas(skill_name: str, db) -> dict:
    """L3: Load input/output schemas on demand."""
    return await db.fetchrow(
        "SELECT input_schema, output_schema FROM skills WHERE skill_name = $1",
        skill_name
    )

```

6. Step 3 — Build the Use Cases Registry

PostgreSQL Schema

```
CREATE TABLE use_cases (  
    use_case_id    UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    use_case_name  VARCHAR(100) UNIQUE NOT NULL,  
    description    TEXT,  
    input_schema   JSONB NOT NULL,  
    flow           JSONB NOT NULL,          -- ordered list of steps (parsed from MD)  
    source_md_file VARCHAR(500),          -- path to originating MD file  
    created_at     TIMESTAMP DEFAULT NOW()  
);
```

Python Use Cases Registry

```
# registry/use_cases_registry.py  
import json  
  
async def upsert_use_case(use_case: dict, db) -> None:  
    """Insert or update a use case parsed from an MD file."""  
    await db.execute("""  
        INSERT INTO use_cases (use_case_name, description, input_schema, flow)  
        VALUES ($1, $2, $3, $4)  
        ON CONFLICT (use_case_name) DO UPDATE  
        SET description = EXCLUDED.description,  
            input_schema = EXCLUDED.input_schema,  
            flow         = EXCLUDED.flow  
    """,  
        use_case["use_case_name"],  
        use_case["description"],  
        json.dumps(use_case["input_schema"]),  
        json.dumps(use_case["flow"]))  
  
    )  
  
async def get_use_case(use_case_name: str, db) -> dict:  
    return await db.fetchrow(  
        "SELECT * FROM use_cases WHERE use_case_name = $1", use_case_name  
    )
```

7. Step 4 — Build the Personas Registry

PostgreSQL Schema

```

CREATE TABLE personas (
  persona_id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  persona_name    VARCHAR(100) UNIQUE NOT NULL,
  description      TEXT,
  system_prompt   TEXT,
  input_schema    JSONB NOT NULL,      -- what the user must provide
  use_cases       JSONB NOT NULL,      -- ordered list of use cases + input mapping
  created_at      TIMESTAMP DEFAULT NOW()
);

```

Seed Persona: QA Agent

```

INSERT INTO personas (persona_name, description, system_prompt, input_schema, use
(
  'qa-agent',
  'QA persona that runs EDR and RRO validation end-to-end and produces a consolid
  'You are a QA orchestrator. Run all assigned use cases and produce a consolidat
  '{
    "required": ["edr_account_number", "rro_property_ids"],
    "properties": {
      "edr_account_number": {"type": "string", "label": "EDR Account Number"},
      "rro_property_ids": {"type": "array", "label": "RRO Property IDs (uploa
    }
  }',
  '[
    {
      "order": 1,
      "use_case_name": "edr-qa",
      "input_mapping": {
        "account_number": "{{ persona_input.edr_account_number }}"
      }
    },
    {
      "order": 2,
      "use_case_name": "rro-qa",
      "input_mapping": {
        "property_ids": "{{ persona_input.rro_property_ids }}"
      }
    }
  ]'
);

```

8. Step 5 — Build the Use Case Orchestrator

The Use Case Orchestrator is Level 2. It reads a use case's flow from PostgreSQL, resolves parameters, builds ADK agents, runs them in sequence, and passes outputs between steps.

```
# orchestrators/use_case_orchestrator.py

import json
from google.adk.agents import LlmAgent
from registry.skills_registry import get_skill_instructions
from registry.tools_registry import get_tool, build_adk_tool
from utils.template_engine import resolve_params
from utils.secrets import resolve_secrets
from orchestrators.agent_orchestrator import AgentOrchestrator

class UseCaseOrchestrator:

    def __init__(self, db):
        self.db = db

    async def run(self, use_case_name: str, input_data: dict) -> dict:
        # Load use case from PostgreSQL
        use_case = await self.db.fetchrow(
            "SELECT * FROM use_cases WHERE use_case_name = $1", use_case_name
        )
        flow = json.loads(use_case["flow"])
        outputs = {}

        for step in sorted(flow, key=lambda x: x["step"]):
            print(f" → Running step {step['step']}: {step['title']}")
            result = await self._execute_step(step, input_data, outputs)
            outputs[step["output_key"]] = result

        return outputs

    async def _execute_step(self, step: dict, input_data: dict, outputs: dict) ->
        # Resolve {{ }} template params
        context = {
            "input": input_data,
            "outputs": outputs,
            "secrets": await resolve_secrets()
        }
        resolved_params = resolve_params(step.get("input_mapping", {}), context)

        # Hand off to Level 3: Agent Orchestrator to handle skill+tool binding
        return await AgentOrchestrator(self.db).run(
            agent_name=f"agent_{step['output_key']}",
            skill_name=step["skill"],
```

```

        tool_name=step.get("tool"),
        resolved_params=resolved_params
    )

def _parse_response(response) -> dict:
    """Extract JSON from ADK agent response."""
    import re
    text = response.text if hasattr(response, "text") else str(response)
    match = re.search(r"\{.*\}", text, re.DOTALL)
    if match:
        try:
            return json.loads(match.group())
        except json.JSONDecodeError:
            pass
    return {"raw_output": text}

```

9. Step 6 — Build the Agent Orchestrator (Agent Factory)

The Agent Orchestrator is **Level 3** — the missing layer. It is responsible for:

1. Loading the skill from the Skills Registry (L2 progressive disclosure)
2. Loading the tool definition from the Tools Registry
3. Binding the resolved runtime parameters to the tool
4. Instantiating a Google ADK agent with the skill instruction + bound tool
5. Executing the agent and returning its output

This is the glue between skills, tools, and the Google ADK runtime. The Use Case Orchestrator calls it once per step — it never builds agents directly.

```

Persona Orchestrator (Level 1) — decides WHICH use cases to run
└─ Use Case Orchestrator (Level 2) — sequences agents within a use case
    └─ Agent Orchestrator (Level 3) — binds skill + tool + params → runs ADK
        ├── Skills Registry → loads instruction (L2)
        ├── Tools Registry → loads tool definition
        └─ Google ADK → LlmAgent(instruction, tools) → execute

```

```
# orchestrators/agent_orchestrator.py
```

```
import json
```

```

from google.adk.agents import LlmAgent
from google.adk.tools import FunctionTool
from registry.skills_registry import get_skill_instructions, get_skill_schemas
from registry.tools_registry import get_tool

class AgentOrchestrator:
    """
    Level 3: Binds a skill to a tool, instantiates a Google ADK agent,
    executes it with resolved parameters, and returns the output.
    Called once per step by the Use Case Orchestrator.
    """

    def __init__(self, db):
        self.db = db

    async def run(
        self,
        agent_name: str,
        skill_name: str,
        tool_name: str | None,
        resolved_params: dict
    ) -> dict:

        print(f"    [AgentOrchestrator] Building agent: {agent_name}")
        print(f"    [AgentOrchestrator]    skill: {skill_name}")
        print(f"    [AgentOrchestrator]    tool:  {tool_name or 'none (LLM reasoning)'}")

        # Step 1: L2 – Load skill instructions from Skills Registry
        skill = await get_skill_instructions(skill_name, self.db)
        if not skill:
            raise ValueError(f"Skill '{skill_name}' not found in skills registry")

        # Step 2: Load tool definition and bind params (if tool is needed)
        adk_tools = []
        if tool_name:
            tool_def = await get_tool(tool_name, self.db)
            if not tool_def:
                raise ValueError(f"Tool '{tool_name}' not found in tools registry")

            # Bind the resolved params to the tool execution function
            bound_tool = self._bind_tool(tool_def, resolved_params)
            adk_tools.append(bound_tool)

        # Step 3: Instantiate Google ADK LlmAgent
        # The skill instruction becomes the agent's system prompt
        agent = LlmAgent(
            name=agent_name,

```



```

        model="gemini-2.0-flash",
        instruction=skill["instruction"],
        tools=adk_tools,
    )

    # Step 4: Build the user message from resolved params
    user_message = self._build_user_message(skill_name, resolved_params, tool)

    # Step 5: Execute the agent
    print(f"    [AgentOrchestrator]    executing...")
    response = await agent.run_async(user_message)

    # Step 6: Parse and return the structured output
    result = self._parse_response(response)
    print(f"    [AgentOrchestrator]    done ✓")
    return result

def _bind_tool(self, tool_def: dict, resolved_params: dict) -> FunctionTool:
    """
    Wraps a tool definition into a callable Google ADK FunctionTool.
    The resolved_params (connection_string, query, url, headers etc.)
    are baked into the closure so the agent can call the tool without
    needing to re-specify them.
    """
    tool_name = tool_def["tool_name"]
    tool_type = tool_def["tool_type"]

    async def execute_tool(**kwargs):
        # Merge baked-in params with any agent-provided overrides
        merged = {**resolved_params, **kwargs}

        if tool_type == "database":
            return await _run_postgres_query(merged)
        elif tool_type == "api":
            return await _run_rest_api(merged)
        else:
            raise ValueError(f"Unknown tool type: {tool_type}")

    return FunctionTool(
        name=tool_name,
        description=tool_def["description"],
        func=execute_tool
    )

def _build_user_message(
    self,
    skill_name: str,

```

```

        resolved_params: dict,
        tool_name: str | None
    ) -> str:
        """Build the user-turn message the agent will receive."""
        if tool_name:
            return (
                f"Execute your task using the '{tool_name}' tool. "
                f"Parameters: {json.dumps(resolved_params, indent=2)}"
            )
        else:
            # No tool – agent uses LLM reasoning on the data directly
            return (
                f"Execute your task using the following inputs: "
                f"{json.dumps(resolved_params, indent=2)}"
            )

def _parse_response(self, response) -> dict:
    """Extract structured JSON from ADK agent response."""
    import re
    text = response.text if hasattr(response, "text") else str(response)
    # Try to extract a JSON block
    match = re.search(r"```json\s*([\s\S]*)```", text)
    if match:
        try:
            return json.loads(match.group(1))
        except json.JSONDecodeError:
            pass
    # Fall back to bare JSON
    match = re.search(r"\{[\s\S]*\}", text)
    if match:
        try:
            return json.loads(match.group())
        except json.JSONDecodeError:
            pass
    # Fall back to array
    match = re.search(r"\[[\s\S]*\]", text)
    if match:
        try:
            return json.loads(match.group())
        except json.JSONDecodeError:
            pass
    return {"raw_output": text}

# -----
# Tool execution helpers (called inside the bound FunctionTool)
# -----

```

```

async def _run_postgres_query(params: dict) -> list[dict]:
    """Execute a PostgreSQL query and return rows as list of dicts."""
    import asyncpg
    conn = await asyncpg.connect(params["connection_string"])
    try:
        rows = await conn.fetch(params["query"], *params.get("params", []))
        return [dict(row) for row in rows]
    finally:
        await conn.close()

async def _run_rest_api(params: dict) -> dict | list:
    """Make an HTTP request and return parsed JSON response."""
    import aiohttp
    method = params.get("method", "GET").upper()
    url = params["url"]
    headers = params.get("headers", {})
    body = params.get("body")

    async with aiohttp.ClientSession() as session:
        async with session.request(
            method, url, headers=headers, json=body,
            timeout=aiohttp.ClientTimeout(total=params.get("timeout_seconds", 30))
        ) as resp:
            resp.raise_for_status()
            return await resp.json()

```

How the Three Levels Work Together

```

# Example: EDR QA Step 1 execution

# Level 2: Use Case Orchestrator calls Level 3
result = await AgentOrchestrator(db).run(
    agent_name = "agent_edr_source_db",
    skill_name = "fetch-source-data",      # → loads SQL instruction from S
    tool_name = "postgres_query",         # → loads tool def from Tools R
    resolved_params = {
        "connection_string": "postgresql://...", # → resolved from secrets
        "query": "SELECT * FROM edr WHERE account = $1", # → from skill definiti
        "params": ["ACC-12345"]             # → resolved from use case input
    }
)

# result = [{"record_id": "R001", "field_a": "X", ...}, ...]

# Level 2 stores result and passes to next step:

```

```
outputs["edr_source_db"] = result
```

10. Step 7 — Build the Persona Orchestrator

The Persona Orchestrator is Level 1. It receives the user's persona selection and inputs, chains use cases together, and runs the final verdict.

```
# orchestrators/persona_orchestrator.py

import json
from orchestrators.use_case_orchestrator import UseCaseOrchestrator
from utils.template_engine import resolve_params
from utils.input_validator import validate_input

class PersonaOrchestrator:

    def __init__(self, db):
        self.db = db

    async def run(self, persona_name: str, user_input: dict) -> dict:
        # Load persona from PostgreSQL
        persona = await self.db.fetchrow(
            "SELECT * FROM personas WHERE persona_name = $1", persona_name
        )
        if not persona:
            raise ValueError(f"Persona '{persona_name}' not found in registry")

        # Validate user input against persona's input schema
        input_schema = json.loads(persona["input_schema"])
        validate_input(user_input, input_schema)

        use_cases = json.loads(persona["use_cases"])
        all_outputs = {}

        print(f"\nExecuting persona: {persona_name}")
        print(f"Running {len(use_cases)} use case(s)...\n")

        # Execute each use case in order
        for uc_config in sorted(use_cases, key=lambda x: x["order"]):
            use_case_name = uc_config["use_case_name"]
            print(f"Use Case [{uc_config['order']}]: {use_case_name}")

            # Resolve input mapping from persona inputs + prior outputs
            uc_input = resolve_params(uc_config["input_mapping"], {
```

```

        "persona_input": user_input,
        "outputs": all_outputs
    })

    # Run use case via Level 2 orchestrator
    uc_result = await UseCaseOrchestrator(self.db).run(
        use_case_name=use_case_name,
        input_data=uc_input
    )
    all_outputs[use_case_name] = uc_result

    # Consolidated verdict across all use cases
    final_verdict = self._consolidate_verdicts(all_outputs)

    return {
        "persona": persona_name,
        "use_case_outputs": all_outputs,
        "final_verdict": final_verdict
    }

def _consolidate_verdicts(self, all_outputs: dict) -> dict:
    """Collect individual verdicts and produce an overall PASS/FAIL."""
    verdicts = []
    for uc_name, outputs in all_outputs.items():
        verdict_key = [k for k in outputs if "verdict" in k.lower()]
        if verdict_key:
            verdicts.append({
                "use_case": uc_name,
                "verdict": outputs[verdict_key[0]]
            })

    overall = "PASS" if all(
        v["verdict"].get("verdict") == "PASS" for v in verdicts
    ) else "FAIL"

    return {
        "overall_verdict": overall,
        "details": verdicts
    }

```

11. Step 8 — End-to-End Data Flow

Here is the complete data flow from the moment a user selects the QA persona to the final verdict:

1. User: selects "qa-agent" persona, uploads Excel with:

edr_account_number = "ACC-12345"

rro_property_ids = ["PROP-001", "PROP-002", "PROP-003"]

2. PersonaOrchestrator.run("qa-agent", user_input)

```
|
|└─ Loads qa-agent persona from PostgreSQL
|└─ Validates user_input against input_schema
|
|└─ USE CASE 1: edr-qa
|  | input_mapping resolves:
|  |   account_number = "ACC-12345"
|  |
|  └─ UseCaseOrchestrator.run("edr-qa", {account_number: "ACC-12345"})
|    |
|    |└─ Step 1: fetch-source-data (postgres_query)
|    |  | input: connection_string, query, params=["ACC-12345"]
|    |  | output: edr_source_db = [{record_id, field_a, field_b, status},
|    |  |
|    |└─ Step 2: fetch-source-data (rest_api_get)
|    |  | input: url="../../records?account=ACC-12345", headers
|    |  | output: edr_source_api = [{record_id, field_a, field_b, status},
|    |  |
|    |└─ Step 3: compare-two-sources
|    |  | input: source_a=edr_source_db, source_b=edr_source_api, key_fie
|    |  | output: edr_comparison = {matched:[], mismatched:[], only_in_a:[
|    |  |
|    └─ Verdict: verdict-summary
|      | input: comparison_result=edr_comparison
|      | output: edr_verdict = {verdict: "PASS", match_pct: 100, summary:
|
|└─ USE CASE 2: rro-qa
|  | input_mapping resolves:
|  |   property_ids = ["PROP-001", "PROP-002", "PROP-003"]
|  |
|  └─ UseCaseOrchestrator.run("rro-qa", {property_ids: [...]}))
|    |
|    |└─ Step 1: fetch-source-data (postgres_query)
|    |└─ Step 2: fetch-source-data (rest_api_get)
|    |└─ Step 3: compare-two-sources
|    └─ Verdict: verdict-summary
|      | output: rro_verdict = {verdict: "FAIL", match_pct: 95.2, summary
|
|└─ PersonaOrchestrator consolidates:
  | overall_verdict = "FAIL" (because rro-qa failed)
  | details = [
```

```
        {use_case: "edr-qa", verdict: "PASS"},
        {use_case: "rro-qa", verdict: "FAIL"}
    ]
```

12. Step 9 — MD File Parser Full Integration

Bootstrap Script: Load All Use Cases from MD Files

```
# scripts/bootstrap_use_cases.py

import asyncio
import asyncpg
import os
from parsers.md_parser import parse_use_case_md
from registry.use_cases_registry import upsert_use_case

USE_CASES_DIR = "use_cases/"

async def main():
    db = await asyncpg.connect(os.environ["DATABASE_URL"])

    for filename in os.listdir(USE_CASES_DIR):
        if filename.endswith(".md"):
            filepath = os.path.join(USE_CASES_DIR, filename)
            print(f"Parsing {filepath}...")
            use_case = parse_use_case_md(filepath)
            await upsert_use_case(use_case, db)
            print(f"    ✓ Loaded: {use_case['use_case_name']}")

    await db.close()
    print("\nAll use cases loaded.")

asyncio.run(main())
```

Template Engine

```
# utils/template_engine.py

import re
import json
from typing import Any
```

```

def resolve_params(params: dict, context: dict) -> dict:
    """
    Recursively resolve {{ }} template expressions in a params dict.
    context = {"input": {...}, "outputs": {...}, "secrets": {...}}
    """
    resolved = {}
    for key, value in params.items():
        resolved[key] = _resolve_value(value, context)
    return resolved


def _resolve_value(value: Any, context: dict) -> Any:
    if isinstance(value, str):
        return _resolve_string(value, context)
    elif isinstance(value, dict):
        return {k: _resolve_value(v, context) for k, v in value.items()}
    elif isinstance(value, list):
        return [_resolve_value(item, context) for item in value]
    return value


def _resolve_string(value: str, context: dict) -> Any:
    pattern = re.compile(r"\{\{\s*(.+?)\s*\}\}")
    matches = pattern.findall(value)

    if not matches:
        return value

    # If the entire string is one template expression, return the raw value
    if len(matches) == 1 and value.strip() == f"{{{ {matches[0]} }}}":
        return _lookup(matches[0], context)

    # Otherwise replace inline
    def replace(m):
        result = _lookup(m.group(1), context)
        return json.dumps(result) if not isinstance(result, str) else result

    return pattern.sub(replace, value)


def _lookup(path: str, context: dict) -> Any:
    """Navigate dot-path like 'outputs.edr_source_db' in context dict."""
    parts = path.split(".")
    val = context
    for part in parts:
        if isinstance(val, dict):
            val = val.get(part)

```



```
        else:
            return None
    return val
```

13. Testing and Validation

Unit Tests

```
# tests/test_md_parser.py

from parsers.md_parser import parse_use_case_md

def test_parse_edr_qa():
    use_case = parse_use_case_md("use_cases/edr_qa.md")
    assert use_case["use_case_name"] == "edr-qa"
    assert len(use_case["flow"]) == 4    # 3 steps + verdict
    assert use_case["flow"][0]["skill"] == "fetch-source-data"
    assert use_case["flow"][2]["skill"] == "compare-two-sources"
    assert use_case["flow"][3]["is_verdict"] == True

def test_parse_rro_qa():
    use_case = parse_use_case_md("use_cases/rro_qa.md")
    assert use_case["use_case_name"] == "rro-qa"
    assert use_case["input_schema"]["required"] == ["property_ids"]
```

Integration Test

```
# tests/test_orchestrator.py

import asyncio
import asyncpg
from orchestrators.persona_orchestrator import PersonaOrchestrator

async def test_qa_persona():
    db = await asyncpg.connect("postgresql://localhost/testdb")
    orchestrator = PersonaOrchestrator(db)

    result = await orchestrator.run(
        persona_name="qa-agent",
        user_input={
            "edr_account_number": "ACC-TEST-001",
            "rro_property_ids": ["PROP-001", "PROP-002"]
```

```

        }
    )

    assert "final_verdict" in result
    assert result["final_verdict"]["overall_verdict"] in ["PASS", "FAIL"]
    assert "edr-qa" in result["use_case_outputs"]
    assert "rro-qa" in result["use_case_outputs"]

asyncio.run(test_qa_persona())

```

Validate Skills Against Agent Skills Standard

```

# Install the skills-ref CLI
npm install -g skills-ref

# Validate each skill directory
skills-ref validate ./skills/fetch-source-data
skills-ref validate ./skills/compare-two-sources
skills-ref validate ./skills/verdict-summary

```

14. Deployment Checklist

Database Setup

```

# Run schema migrations
psql $DATABASE_URL -f db/schema.sql

# Seed tools
psql $DATABASE_URL -f db/seed_tools.sql

# Seed skills
psql $DATABASE_URL -f db/seed_skills.sql

# Seed personas
psql $DATABASE_URL -f db/seed_personas.sql

```

Load Use Cases from MD Files

```
python scripts/bootstrap_use_cases.py
```

Environment Variables

```
DATABASE_URL=postgresql://user:pass@host:5432/qa_platform
SECRETS_MANAGER=gcp # or aws, azure, vault
GCP_PROJECT_ID=your-project-id
```

```
# These go into your secrets manager, not env vars:
# edr_db_connection, edr_api_token
# rro_db_connection, rro_api_token
```

Adding a New Use Case (Zero Code Changes)

```
# 1. Write the use case MD file
vim use_cases/new_qa.md

# 2. Load it into PostgreSQL
python scripts/bootstrap_use_cases.py

# 3. Add it to the relevant persona (SQL update)
# UPDATE personas SET use_cases = use_cases || '{"order": 3, "use_case_name": "n
# WHERE persona_name = "qa-agent";

# Done — no code changes required
```

Reusability Summary

Component	Reused By	How
<code>postgres_query</code> tool	Any skill needing DB access	Referenced by <code>tool_name</code> in skill
<code>rest_api_get</code> tool	Any skill needing API access	Referenced by <code>tool_name</code> in skill
<code>fetch-source-</code> <code>data</code> skill	EDR use case, RRO use case, any future use case	Referenced by <code>skill</code> in use case MD
<code>compare-two-</code> <code>sources</code> skill	EDR use case, RRO use case, any future use case	Referenced by <code>skill</code> in use case MD
<code>verdict-summary</code> skill	All QA use cases	Referenced as final step in every use case

edr-qa use case	QA persona, any future persona needing EDR	Referenced in persona's use_cases array
rro-qa use case	QA persona, any future persona needing RRO	Referenced in persona's use_cases array